# Chapter 14

# A Catalog of Story Smells

This chapter will present a catalog of "bad smells," that is, indicators that something is amiss in a project's application of user stories. Each smell will be described and one or more solutions provided.

## Stories Are Too Small

**Symptom:** A frequent need to revise estimates.

**Discussion:** Small stories often cause problems with the estimating and scheduling of stories. This happens because the estimate assigned to a small story can change dramatically depending on the order in which the story is implemented. For example, consider these two small stories:

- Search results may be saved to an XML file.
- Search results may be saved to an HTML file.

There is clearly a great deal of overlapping work between these two stories. Time spent on one of the stories will reduce the time spent on the other. Stories like these should be combined for planning purposes. When the story is slotted into an iteration the story can be split; but, until it is necessary, the stories should remain combined.

## Interdependent Stories

**Symptom:** Difficulty planning iterations because of dependencies between stories.

**Discussion:** When two or more stories are dependent upon one another, it becomes difficult to plan the individual stories into iterations. The team finds itself in a situation where a particular story may only be added to an iteration if

another story is also added to the iteration. But that story may only be added if a third story is also added, and so on. This is caused when stories are either too small or have been inappropriately split.

If you suspect that stories are too small, the easy solution is to simply combine the interdependent stories into one. If, instead, the stories appear for all other purposes to be appropriately sized, then look at how the interdependent stories have been separated. Chapter 7, "Guidelines for Good Stories," offered the suggestion that stories be split so as to be a full "slice of cake," including functionality from all layers of the application.

## Goldplating

**Symptom:** Developers are adding features that were not planned for the iteration, or are interpreting stories liberally and going beyond what is necessary to implement a story.

**Discussion:** *Goldplating* refers to the addition of unnecessary features by the developers. Some developers have a tendency to add enhancements or to go beyond what is needed to satisfy a customer's stated needs. This can happen for a couple of reasons. First, some developers like "wowing" the customer and that is harder to do with the greater customer involvement in agile processes. If the customer is involved day–to–day it is hard to give her a pleasant surprise and get the "wow" some developers enjoy.

Second, when working on agile, story-driven projects with short iterations, developers typically feel a great deal of pressure to always be producing. Goldplating allows them a brief chance to escape this pressure. After all, if the feature can't be finished in time, no one will even know it had been started.

Finally, developers enjoy putting their own mark on a project, and adding a few pet features is one way for them to do this.

**An Example of Goldplating**

On one project I was on we had a story to take a very crowded screen and rewrite it as a tabbed dialog to improve usability. After the developer finished this change, he enhanced the low-level tab dialog code in this application so that a tab could be torn from its current location and moved about the screen. This was not something the customer asked for. Developers need to stay focused on the stories prioritized by the customer. If a developer has a good idea for a new story, she should suggest it to the customer for possible inclusion in the next iteration.

If a project is experiencing significant goldplating by developers, it can be prevented by increasing the visibility of the tasks that everyone is working on. For example, hold brief daily meetings where everyone says what he or she is working on. With greater visibility into what each developer is working on the team will self-police against gold-plating.

Similarly, end–of–iteration review meetings where all of the new functionality is demonstrated in detail for the customer and other stakeholders will help identify goldplating that occurred during the iteration. It will be too late to correct it during that iteration but the team will be more aware of it for future iterations.

Finally, if the project has a QA organization they can also help identify goldplating, especially if they were involved in the conversations between the programmer and the customer.

## Too Many Details

**Symptom:** Too much time is being spent gathering details well in advance of a story being implemented. Or, more time is spent writing about stories than talking about them.

**Discussion:** One of the benefits to writing stories on notecards is that the space available for writing the story is quite limited. It is hard to cram lots of detail onto a small card. Including too many details in a story is indicative of placing too much value on documentation and favoring it over conversation.

Tom Poppendieck (2003) has made the observation that "If you run out of room, use a *smaller* card." This is a great idea because it forces story writers to very consciously include fewer details in the stories.

## Including User Interface Detail Too Soon

**Symptom:** Stories written early in a project (especially a project to develop a new product) that include detail about the user interface.

**Discussion:** At some point on a project the team will definitely write user stories with very direct assumptions (or even direct knowledge) about the user interface. For example, "A Job Seeker can view information about the hiring company from the Job Description page." However, for as long as possible you should avoid writing stories with this level of detail.

Early in the project you do not know that there will be a "Job Description page" so avoid constraining the project by associating stories with it. Instead of the story above, write one such as "When viewing details about a job, a Job Seeker may view information about the hiring company."

## Thinking Too Far Ahead

**Symptom:** Indicators of this smell may be that stories are hard to fit on note cards, there is interest in using a software system instead of note cards that isn't driven by team size or location, someone proposes a story template to capture all of the details necessary for a story, or perhaps that a suggestion is made to give estimates in finer precision (for example, hours instead of days).

**Discussion:** This smell is particularly common among teams accustomed to large upfront "requirements engineering" efforts. For a team to overcome this smell they may need a refresher course on the strengths of stories. Fundamental to the use of stories is the recognition that for most problems it is impossible to identify all requirements in advance. Good software emerges through repeated iterations in which increasing amounts of detail are added to the software. Stories fit well with this approach because of the ease with which detail can be expressed in later versions of a story. The team may need to remind itself what it was about their prior development process that led them to adopt stories.

## Splitting Too Many Stories

**Symptom:** Frequently splitting stories during iteration planning so that the right amount of work fits in the iteration.

**Discussion:** When the developers and customer select the stories they will move into an iteration, they will sometimes need to split a story into two or more constituent stories. Typically a story needs to be split during planning for one of two reasons:

1. The story is too large to fit into the iteration.
2. The story contains both high and low sub-stories, and the customer only wants the high priority sub-stories done during the coming iteration.

Neither of these cases is representative of a problem. Many projects and teams will have occasions when it will be useful to split a story to accommodate

the duration of a sprint and fit with the team's observed velocity. However, the situation begins to smell when the team finds itself splitting stories frequently.

If stories are being split more often than feels reasonable, then consider taking a pass through the remaining stories and looking for stories that should be split.

## Customer Has Trouble Prioritizing

**Symptom:** Choosing among and prioritizing stories is often difficult; but sometimes prioritizing the stories is so difficult that it can be considered a smell.

**Discussion:** If a customer is having trouble prioritizing stories, the first thing to consider is the size of the stories. If stories are too large, they can be difficult to prioritize. Suppose, at the extreme, the BigMoneyJobs website included only the following three stories:

- A Job Seeker can search for jobs.
- A Company can post a job opening.
- A Recruiter can search for candidates.

Pity the poor customer who has to prioritize only among these stories! Her reaction is probably to say, "But can't I have a little of this and some of that?" In this case, get rid of the current stories and replace them with smaller ones so that she can select the pieces of each that she wants.

Also, it may be difficult to prioritize stories if they have not been written to express business value. For example, suppose the customer is presented with these stories:

- A user connects to the database via a connection pool.
- A user can view detailed error information in a log file.

Stories like these can be very difficult for the customer to prioritize because the business value of each is not clear. The stories should be rewritten so that the value of each is clear to the customer. How each is rewritten will vary depending on the customer's technical knowledge, which is why the best recommendation is to have the customer write the stories herself. For example, consider these rewritten versions of the preceding stories:

- A user can start the application without a noticeable lag while connecting to the database.
- Whenever an error occurs, users are given enough information to know how to correct the error.

## Customer Won't Write and Prioritize the Stories

**Symptom:** The customer on a project will not accept responsibility for writing or prioritizing the stories.

**Discussion:** In a blame-filled organization there are always some individuals who have learned that their best decision is to avoid all responsibility. If you're not responsible for something you can't be blamed for it when it fails, yet there's usually a way to lay claim to at least some of the success. Individuals in this type of culture will want nothing to do with hard decisions like prioritizing features into and out of releases. They'll fall back on statements such as "It's not my problem that you can't complete everything by the deadline, figure out a way to do it."

The best solution I've found in these situations is to find a way to let the customer off the hook. I find a non-threatening way for her to express her opinions. Depending on the individual, this may have to be a private conversation. If I'm working with multiple customers, I tell them each that I'm collecting input from others but that I'm the one who will be responsible for the final decisions, especially if they turn out to be wrong.

## Summary

In this chapter we learned about the following smells:

- stories that are too small
- interdependent stories
- goldplating
- adding too many details to stories
- including user interface details too soon
- thinking too far ahead

- splitting too many stories
- trouble when prioritizing stories
- customer won't write and prioritize stories

## Developer Responsibilities

- You share a responsibility with the customer to be aware of these smells—as well as any others you may detect—and then to work to correct any that affect your project.

## Customer Responsibilities

- You share a responsibility with the developers to be aware of these smells—as well as any others you may detect—and then to work to correct any that affect your project.

## Questions

14.1 What should you do if the team is consistently finding it difficult to plan the next iteration?

14.2 What should the team do if they are consistently running out of room to write on the story cards?

14.3 What could cause the customer to have a difficult time prioritizing stories?

14.4 How do you know if you are splitting too many stories?

*This page intentionally left blank*